Sharpening Where There Is Headroom: What GRPO Changes in Text-to-SQL Across Model Scale

Ved Chadderwala
University of Southern California
chadderw@usc.edu

Working draft, October 25, 2026
Code and results: https://github.com/ved-2004/RLVR-Text-to-SQL

1 Introduction and Motivation

Reinforcement learning from verifiable rewards (RLVR) has become a standard post-training step for reasoning and code models [2, 5, 9]. The recipe is simple: sample several answers, check each with a program, and push the model toward the ones that pass. What is less clear is *what* this changes. A model that gets more questions right after RL could have learned to solve new problems, or it could have learned to pick, on the first try, an answer it was already able to produce some of the time. Yue et al. [12] argue that on math and code benchmarks it is mostly the second: RL raises pass@1 but does not raise, and can lower, pass@k for large k.

This report studies that question on text-to-SQL, which is a useful instrument for three reasons. First, the verifier is cheap and objective: execute the query and compare result sets. Second, the verifier has a known and measurable false-positive rate, because a wrong query can return the right rows on one database by coincidence [13]. This lets us check whether the policy learns to exploit the reward. Third, SQL tokens carry explicit roles (keywords, aggregates, table names, aliases, literals), so token-level changes can be grouped by what they do.

We train Qwen2.5-Coder-Instruct at 1.5B and 7B parameters [4] with one fixed GRPO recipe and a binary execution reward on the Spider training set [11], then ask four questions:

1. How much does the same recipe help at each scale, and what predicts the size of the gain?
2. Where in the output distribution does the model move, and does that differ by scale?
3. Does RL change *how* the model writes SQL, not only whether it is correct?
4. Does the policy learn to exploit the verifier's false positives?

Our main findings are:

- The same recipe raises execution accuracy by +12.7 pp at 1.5B (59.1% to 71.8%) and by +2.6 pp at 7B (76.8% to 79.4%). The size of the gain lines up with the base model's pass@1 to pass@8 gap (18.8 vs. 6.8 points). This is a two-point observation, not a fitted law.
- The 1.5B model moves 3.65× further from its base in per-token KL and fixes 155 dev questions versus 35 for the 7B. The 1.5B change is broad; the 7B change is concentrated on aggregate and `FROM`-clause tokens.

- Without any style term in the reward, the 1.5B model drops table aliases (2.63 to 0.36 per query) and switches to `table.column` references, which is how the untrained 7B already writes. It moves *away* from the style of the gold queries. Forcing the style does not reproduce the gain (E3, E4), but execution voting over 8 base-model samples recovers 86% of it (E3).
- The single-database vs. test-suite gap stays between 6 and 9 points. It narrows at 7B and widens slightly at 1.5B, so we see no clear sign of reward hacking, but also no sign that RL removes the false positives.

2 Background and Related Work

GRPO and RLVR. Group Relative Policy Optimization [9] drops the value network of PPO. For each prompt it samples a group of G completions, scores them, and uses the group-normalized reward as the advantage:

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}. \tag{1}$$

A KL penalty with coefficient β keeps the policy near a reference model. When every completion in a group gets the same reward, all advantages are zero and the prompt contributes no gradient. DAPO [10] addresses this with dynamic sampling, which filters out such groups. RLVR with GRPO-style updates is used in DeepSeek-R1 [2] and Tülu 3 [5].

Does RL add capability? Yue et al. [12] compare base and RL-trained models with pass@k [1] at large k and find that base models match or beat RL models once k is large, which suggests RL mostly reweights solutions already in the base model's distribution. Mukherjee et al. [7] find that RL updates only a small subnetwork of parameters, consistent with a light-touch change. Our scale comparison asks a related question from a different side: if RL mostly converts existing pass@k into pass@1, the room available to convert should predict the gain.

RL for text-to-SQL. SQL-R1 [6] trains reasoning models with GRPO and execution-based rewards. Reasoning-SQL [8] adds partial rewards for schema linking, syntax, and n-gram similarity. Both target state-of-the-art accuracy. We instead keep the recipe minimal (binary reward, no reasoning trace, LoRA) so that changes can be attributed and compared across scale.

Evaluating SQL by execution. Spider [11] reports exact set match and execution accuracy. Execution accuracy has false positives: a query such as `WHERE age > 34` may return the same rows as the gold `WHERE age >= 34` on one database. Test-suite accuracy [13] runs each query on a distilled set of perturbed databases and counts it correct only if it matches gold on all of them. Because our reward is execution on a single database, the gap between single-database and test-suite accuracy measures how much of the reward is earned by coincidence.

3 Experiments and Results

3.1 Setup

Models. Qwen2.5-Coder-1.5B-Instruct and Qwen2.5-Coder-7B-Instruct [4]. These are general code models, not SQL-specialized, so that a gain from RL is not confounded with prior SQL fine-tuning.

Training. GRPO via TRL `GRPOTrainer` with LoRA [3] ($r = 16$, $\alpha = 32$, all linear layers), $G = 8$ completions per prompt, learning rate 10^{-5}, $\beta = 0.04$, sampling temperature 0.9. Training data is the full Spider train set (7,000 examples). [TODO: epochs, batch size, max completion length, wall-clock and GPU type]. The prompt gives the schema as one line per table and asks for a single SQLite query in a `sql` code block. There is no reasoning trace.

Reward. Binary execution reward: extract the query, run it read-only on the example's SQLite database with a 5 s timeout, and return 1 if its result set equals the gold result set, else 0. Write statements are rejected.

Evaluation. Full Spider dev set (1,034 questions), greedy decoding with vLLM, identical prompt for all four models. We report:

- **EX**: execution accuracy from the test-suite evaluator [13] run on the original dev databases.
- **TS**: the same evaluator run on the distilled test suite. Because only the databases change, **EX** − **TS** (the FP gap) isolates false positives.
- **EX (official)**: the original Spider evaluator, for comparison.
- **pass@1 / pass@8**: from 8 samples per question, scored with a set-based result comparison. [TODO: sampling temperature for pass@k].
- **Set-op EX**: execution accuracy on the 80 dev questions whose gold query uses `INTERSECT`, `UNION` or `EXCEPT`.
- **ECE**: expected calibration error. [TODO: state exactly what confidence is binned].

Evaluation note. Two earlier versions of these numbers were wrong. The first pipeline truncated long prompts at 1,536 tokens and reported a 1.5B base EX of 34.5%. The second saved only the last 2,500 characters of the official evaluator's log, which cut off the execution row; the value we read as EX was the SELECT-clause partial-match recall. All numbers in this report are generated from the stored result files by `report/make_numbers.py`, which reads each metric by its row in the evaluator output.

3.2 Main results

GRPO helps the 1.5B model on every execution metric by about 12 to 13 points and the 7B by about 2 to 3.5 (Table 1). The untrained 7B still beats the trained 1.5B on EX (0.768 vs. 0.718). Calibration gets slightly worse at both scales.

3.3 The gain tracks sampling headroom

In both models pass@1 rises more than pass@8 (Table 2, Figure 1), and the EX gain closely matches the pass@1 gain. Most of the improvement is turning questions the model sometimes solved into questions it solves on the first try. The model with more of these questions available (1.5B) gains more.

This is not shown to be pure sharpening: pass@8 also rises at both scales. Whether the 1.5B learns set operations it could not do before is measured by the set-op row of Table 1. With two models and one seed, we read Table 2 as consistent with "gain scales with headroom," not as evidence for it.

Table 1: Spider dev results. Δ is in percentage points.

	1.5B			7B		
Metric	Base	GRPO	Δ	Base	GRPO	Δ
EX	0.591	0.718	+12.7	0.768	0.794	+2.6
TS	0.510	0.632	+12.2	0.695	0.730	+3.5
FP gap (pts)	8.1	8.6	+0.5	7.3	6.4	−0.9
EX (official)	0.419	0.492	+7.3	0.507	0.520	+1.3
pass@1	0.566	0.689	+12.3	0.729	0.755	+2.6
pass@8	0.753	0.786	+3.3	0.797	0.816	+1.9
Set-op EX	0.350	0.588	+23.8	0.700	0.675	−2.5
ECE	0.049	0.062		0.074	0.086	

Table 2: Headroom before training vs. gain after training (points).

Model	pass@8 − pass@1 (base)	Δ pass@1	Δ pass@8	Δ EX
1.5B	18.8	+12.3	+3.3	+12.7
7B	6.8	+2.6	+1.9	+2.6

3.4 Where the model moves: token-level divergence

We teacher-force the gold query through each base and GRPO model and compute, per token, the KL divergence between their next-token distributions, the change in entropy, and the change in gold-token log-probability. Tokens are tagged by SQL role (keyword, aggregate, table, column, alias, literal, punctuation).

- **Magnitude.** Mean per-token KL is 0.043 nats at 1.5B and 0.012 at 7B, a 3.65× difference. GRPO fixes 155 dev questions at 1.5B (wrong before, right after) and 35 at 7B. [TODO: report questions broken (right before, wrong after) at each scale].
- **Shape at 1.5B.** Entropy drops by about 0.11 nats with no strong preference for any token role.
- **Shape at 7B.** On the questions GRPO fixed, KL is concentrated on aggregate tokens (17.3× their share of tokens) and on `FROM`-clause tokens (5.2×).

One reading is that the 1.5B model is still learning general habits for this task, while the 7B already has them and only adjusts a few decisions.

3.5 Surface-form convergence

The reward only checks the result set. Even so, GRPO drives the 1.5B model to drop the alias convention used in Spider's gold queries and to write qualified `table.column` references instead, close to how the untrained 7B already writes. The 7B's style does not change. The 1.5B also uses fewer joins, which matters because unneeded joins are a common error for this model. Whether this style shift *causes* part of the gain is tested in E3 and E4 (Sections 3.9 and 3.10).

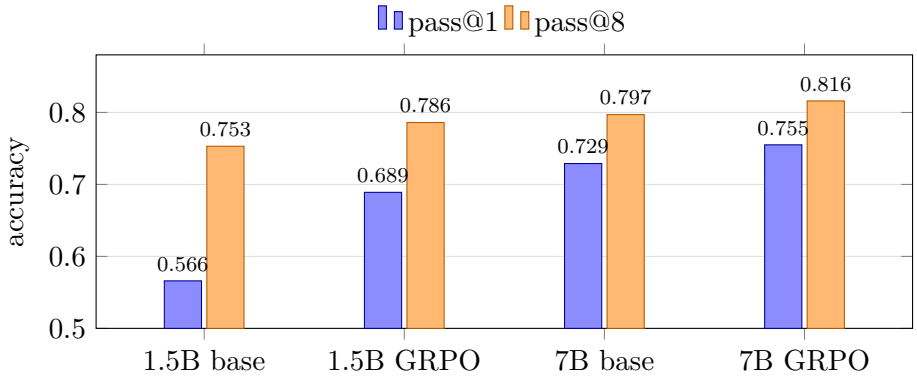


Figure 1: pass@1 rises much more than pass@8, and the rise is largest where the gap between them was largest.

Table 3: Surface style of generated SQL, per query on Spider dev.

	Gold	1.5B base	1.5B GRPO	7B base	7B GRPO
Aliases (`T1`, `T2`, ...)	2.93	2.63	0.36	0.43	0.45
Qualified `table.column`	[TODO:]	0.46	1.41	1.62	1.59
`JOIN`	[TODO:]	0.51	0.35	[TODO:]	[TODO:]

3.6 False positives and reward hacking

A binary execution reward on one database pays for coincidentally correct queries, so a policy could learn to produce them. If it did, the EX − TS gap should grow during training. It moves by +0.5 points at 1.5B and −0.9 at 7B (Table 1). Within this recipe we see no strong reward hacking, but the false-positive rate is large enough that it should be reported next to any execution-accuracy gain.

3.7 Training dynamics

A mean of 0.625 of GRPO groups had zero reward variance (`frac_reward_zero_std`): all 8 completions were right, or all were wrong, so they gave no gradient. Most of the compute went to prompts that were already solved or still out of reach. This is the case dynamic sampling in DAPO [10] is designed for.

3.8 Limitations

Two model sizes, one seed per configuration, one dataset, one reward. The headroom observation rests on two points. pass@k uses a looser result comparison than the evaluators. LoRA limits how far the policy can move, so some differences between scales could come from LoRA capacity rather than RL itself.

3.9 E3: style prompts and execution voting at 1.5B

We decode the 1.5B base and GRPO models with three prompts: the default prompt, one that forbids aliases, and one that requires `T1`, `T2` aliases. Each is run greedily and with execution voting:

8 samples at temperature 0.8, executed, and the most common result set wins (no gold). Generation uses vLLM, as in the main evaluation.

Table 4: E3 on Spider dev at 1.5B. "Vote" is majority voting over 8 samples by result set. Aliases are per query.

Model	Prompt, decoding	EX	TS	EX (official)	Aliases	`table.column`
Base	default, greedy	0.591	0.510	0.418	2.65	0.45
	default, vote	0.697	0.597	0.464	2.21	0.69
	no alias, greedy	0.585	0.515	0.426	2.43	0.45
	no alias, vote	0.712	0.607	0.511	1.74	0.87
	alias, greedy	0.432	0.366	0.224	6.28	0.00
	alias, vote	0.617	0.502	0.347	5.87	0.02
GRPO	default, greedy	0.714	0.629	0.489	0.37	1.40
	default, vote	0.744	0.641	0.499	0.36	1.45
	no alias, greedy	0.730	0.636	0.502	0.31	1.44
	no alias, vote	0.746	0.656	0.526	0.35	1.45
	alias, greedy	0.627	0.566	0.420	5.48	0.01
	alias, vote	0.709	0.626	0.467	5.27	0.02

Voting recovers most of the GRPO gain. With the default prompt, execution voting raises the base model's EX by +10.6 pp. That is 86% of the greedy base-to-GRPO gap on EX and 73% on TS. Combined with the no-alias prompt, it reaches 98% on EX and 82% on TS. Voting on the GRPO model adds only +3.0 pp. At 1.5B, most of what GRPO does can be done at inference time by sampling and voting with no gold, which agrees with the pass@k results: RL mostly moves answers the model could already produce to the first try.

The prompt does not change the base model's style. Forbidding aliases moves the base model from 2.65 to 2.43 aliases per query, and EX by −0.6 pp. The instruction mostly does not take, so E3 cannot test whether `table.column` style alone would help the base model. Voting removes more aliases than the instruction does, which is weak evidence that alias-free queries are more often correct.

Forcing aliases hurts both models. The alias prompt changes EX by −15.9 pp for the base model and −8.7 pp for GRPO. Even with aliases forced, GRPO (0.627) stays above the base model with its default prompt (0.591). So the GRPO gain is not only a change of style, but the trained model is also not indifferent to style.

3.10 E4: alias neurons at 1.5B

Contrast. On Spider *train* questions (dev stays clean), the base model writes each query twice, once told to use aliases and once told not to. We keep the 139 pairs (plus 32 held out) where both versions are correct by execution, the alias version has at least two alias mentions and the other has none. Under the original prompt, alias preference is $m = \log p(\text{alias version}) - \log p(\text{qualified version})$. The mean is −5.36 for the base model and −11.07 for GRPO, and the alias version is preferred on 5.0% and 0.7% of pairs. m is a whole-sequence log-probability and is not

length-normalized. The alias version is usually longer, so the sign of m is not meaningful on its own, but the change of -5.72 from base to GRPO is.

Attribution. Attribution patching (AtP) from base to GRPO MLP activations over all 250,880 MLP neurons predicts a total margin change of -7.48, against a real change of -5.72. The top 100 neurons account for 10% of the AtP total, so the shift is spread over many neurons. Of the top 50 neurons by AtP, 17 are in layer 27, the last layer. Several top neurons project onto `T` tokens under the logit lens, for example L27 N3289 (AtP rank 5). We screened 40 candidates by single-neuron steering on 32 held-out pairs. For 21 of them GRPO moved the activation in the direction that lowers m, about what chance would give.

Table 5: E4 steering and ablation of the 1.5B base model on Spider dev (HF greedy decoding). Dose is the norm of the vector added to the residual stream, in units of the median residual norm at that layer. "Valid" is the fraction of predictions that execute.

Run	EX	TS	Valid	Aliases	`table.column`
Base	0.592	0.511	0.784	2.62	0.45
GRPO	0.720	0.633	0.910	0.34	1.41
Zero-ablate top 8	0.620	0.541	0.811	1.32	1.00
Steer top 2, dose 0.5	0.463	0.394	0.633	0.00	1.93
Steer top 2, dose 1	0.000	0.000	0.000	0.00	0.00
Steer top 8, dose 0.25	0.137	0.133	0.165	0.00	0.50
Steer top 8, dose 0.5	0.000	0.000	0.000	0.00	0.00
Random 2, dose 1	0.000	0.000	0.000	0.00	0.00
Random 8, dose 0.5	0.000	0.000	0.000	0.00	0.00
Top 2 reversed, dose 1	0.000	0.000	0.000	0.00	0.00
GRPO, top 2 reversed, dose 1	0.000	0.000	0.000	0.00	0.00

Steering. The steering doses are too large. 6 of the 8 steered runs, including both random controls, produce no executable SQL, so the random controls tell us nothing. The one usable steered run (top 2, dose 0.5) removes aliases completely but changes EX by -12.9 pp, together with a fall in valid SQL. Removing the style by force does not bring the GRPO gain.

Ablation. Zero-ablating the top 8 screened neurons halves the base model's aliases and raises EX by $+2.8$ pp and TS by $+3.0$ pp, which is 22% of the base-to-GRPO gap on EX. There is no random-ablation control yet, so we do not claim that these neurons carry part of the gain.

Reading E3 and E4 together. The GRPO shift away from aliases is spread over many neurons, with a cluster in the last layer. Forcing the style in either direction does not reproduce the GRPO gain, while selection by voting reproduces most of it. Style looks like a side effect of sharpening, not the mechanism.

3.11 Next experiments

1. **E4 at smaller doses.** Repeat the steering sweep at doses where random neurons leave the model intact, and add a random zero-ablation control for the top-8 ablation. Normalize m

by length so that pairs of different lengths are comparable.

2. **Robustness.** Two more seeds at 1.5B, and DAPO at 1.5B to test whether removing zero-variance groups changes the outcome.

References

[1] Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[2] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[4] Binyuan Hui et al. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

[5] Nathan Lambert et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

[6] Peixian Ma et al. Sql-r1: Training natural language to sql reasoning model by reinforcement learning. *arXiv preprint arXiv:2504.08600*, 2025.

[7] Sagnik Mukherjee, Lifan Yuan, Dilek Hakkani-Tür, and Hao Peng. Reinforcement learning finetunes small subnetworks in large language models. *arXiv preprint arXiv:2505.11711*, 2025.

[8] Mohammadreza Pourreza et al. Reasoning-sql: Reinforcement learning with sql tailored partial rewards for reasoning-enhanced text-to-sql. *arXiv preprint arXiv:2503.23157*, 2025.

[9] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[10] Qiying Yu et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[11] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. *arXiv preprint arXiv:1809.08887*, 2018.

[12] Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.

[13] Ruiqi Zhong, Tao Yu, and Dan Klein. Semantic evaluation for text-to-sql with distilled test suites. *arXiv preprint arXiv:2010.02840*, 2020.